

Experiment -1

Linux Programming Environment & Basic Commands

A) Setting up of Linux Programming Environment

We can do C programming online or offline according to the needs. It is required for us to keep working on a Linux Environment.

Online:

Cocalc.ai – Brief the steps followed?

Steps:

1. Visit [CoCalc](#) and create an account
2. Create a new project
3. Open the project and create a Linux terminal
4. Create a C source file
5. Enter the program
6. Save the file and exit the editor
7. Compile it using GCC
8. Run the executable

Offline:

Methods: Dual boot, Single boot, Bootable Pendrive, VM

Brief the methods?

1. Dual boot

STEPS:

1. Back up important files.
2. Download a Linux ISO, such as Ubuntu.
3. Create a bootable USB drive.
4. Free some disk space for Linux.
5. Boot from the USB drive.
6. Select **Install alongside Windows**.
7. Complete the installation.
8. Choose Linux or Windows from the boot menu whenever the computer starts.

2. Single boot

STEPS:

1. Back up all important files.
2. Create a bootable Linux USB drive.
3. Boot the computer from the USB.
4. Select **Erase disk and install Linux**.
5. Complete the installation.
6. Install C development tools

`$sudo apt update`

`$sudo apt install build-essential`

3. Bootable pendrive

STEPS:

1. Download a Linux ISO.
2. Use Rufus, Balena Etcher, or a similar utility to create a bootable USB.
3. Restart the computer and open its boot menu.
4. Select the USB device.
5. Choose **Try Linux**.
6. Open the terminal and work in the live Linux environment.

4. Virtual machine

1. Install VirtualBox, VMware, or another virtualization program.
2. Download a Linux ISO.
3. Create a new virtual machine.
4. Assign RAM, processor cores, and virtual-disk space.
5. Attach the Linux ISO and start the virtual machine.
6. Install Linux inside it.
7. Install GCC and related tools:

\$sudo apt update

\$sudo apt install

1. Virtual Machine Manager (Hypervisor)

Type-1 (Baremetal)

What is this?

Ans- A Type-1 hypervisor, also called a bare-metal hypervisor, is virtualization software that runs directly on a computer's physical hardware. It does not run inside a regular host operating system such as Windows.

E.g. : VMware, Xen, KVM etc.

Type-2 (Hosted)

What is this?

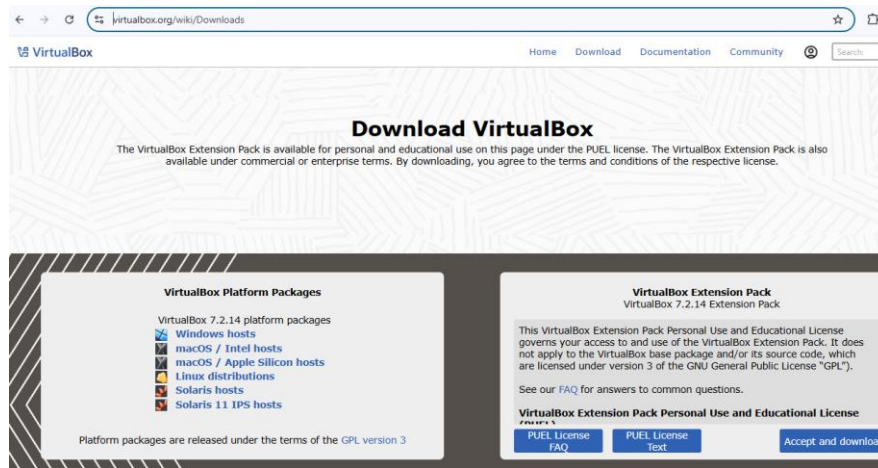
Ans- A Type-2 hypervisor, also called a hosted hypervisor, is virtualization software installed as an application on an existing operating system, such as Windows, Linux, or macOS. They are commonly used for learning, software testing, and running another operating system without changing or removing the main operating system.

E.g.- Oracle Virtualbox, VM Ware.

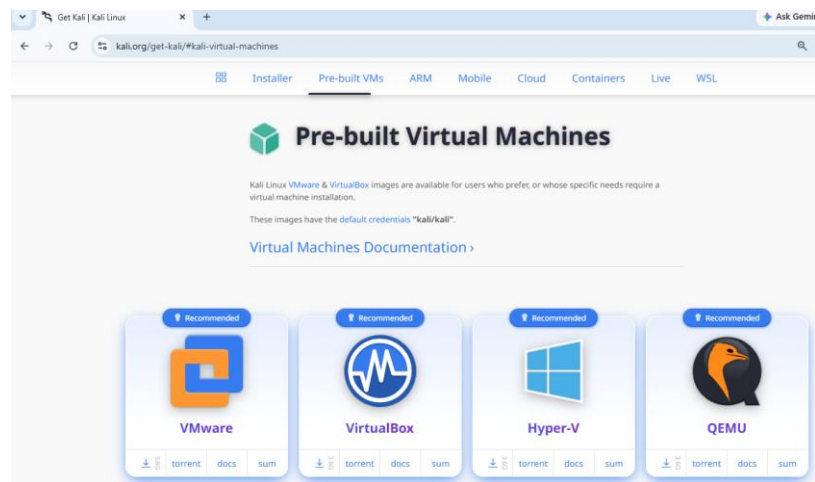
We would be using Oracle Virtualbox.

2. Installation of VMM & VM

Step-1: Download Oracle Virtual Box for "Windows hosts"

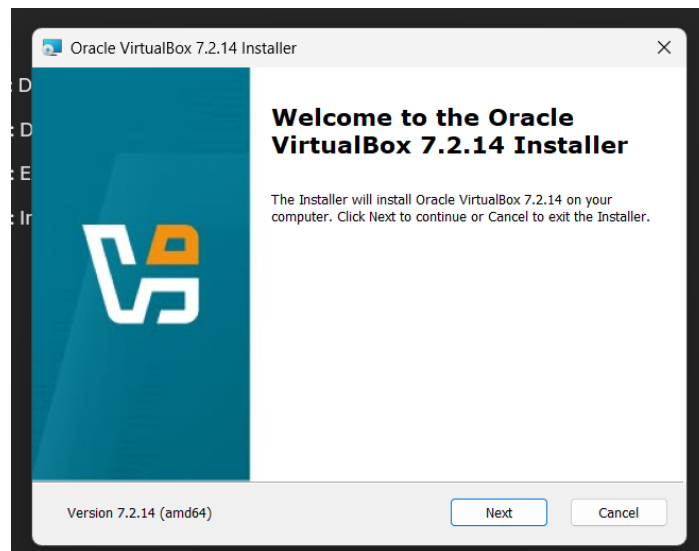


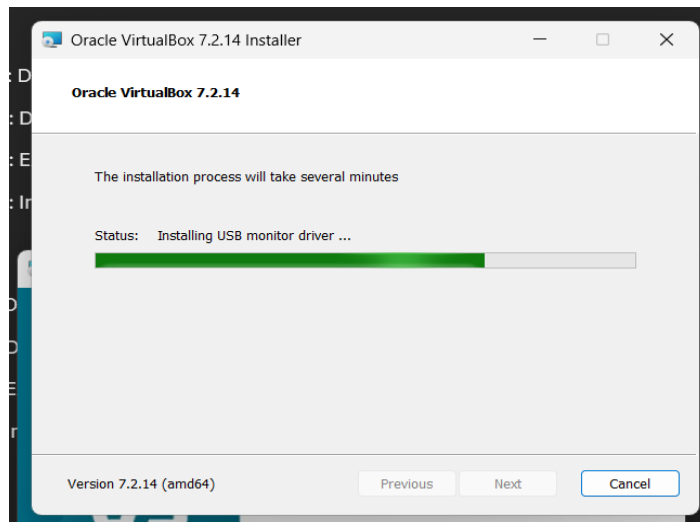
Step-2: Download Pre-built Kali Linux VM for Virtualbox



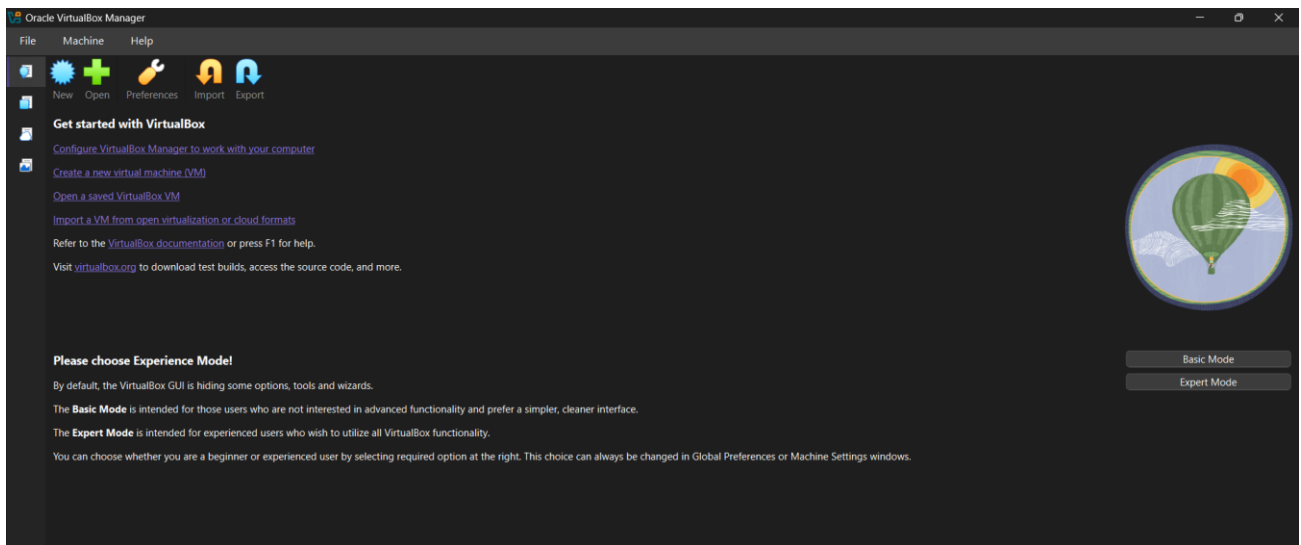
Step-3: Extracted the Kali Linux downloaded file

Step-4: Install Virtualbox

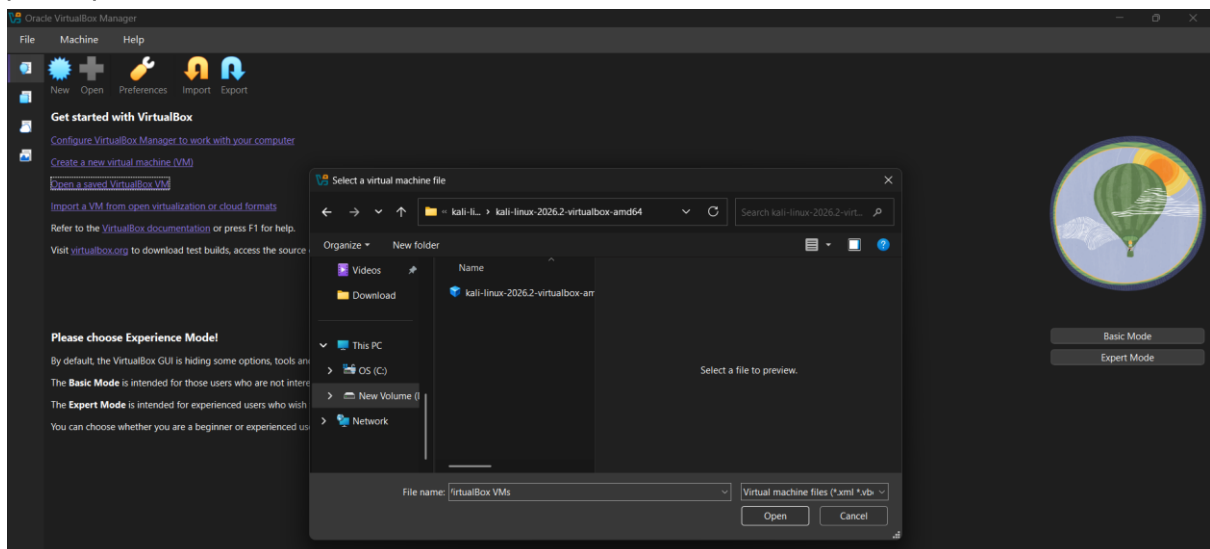


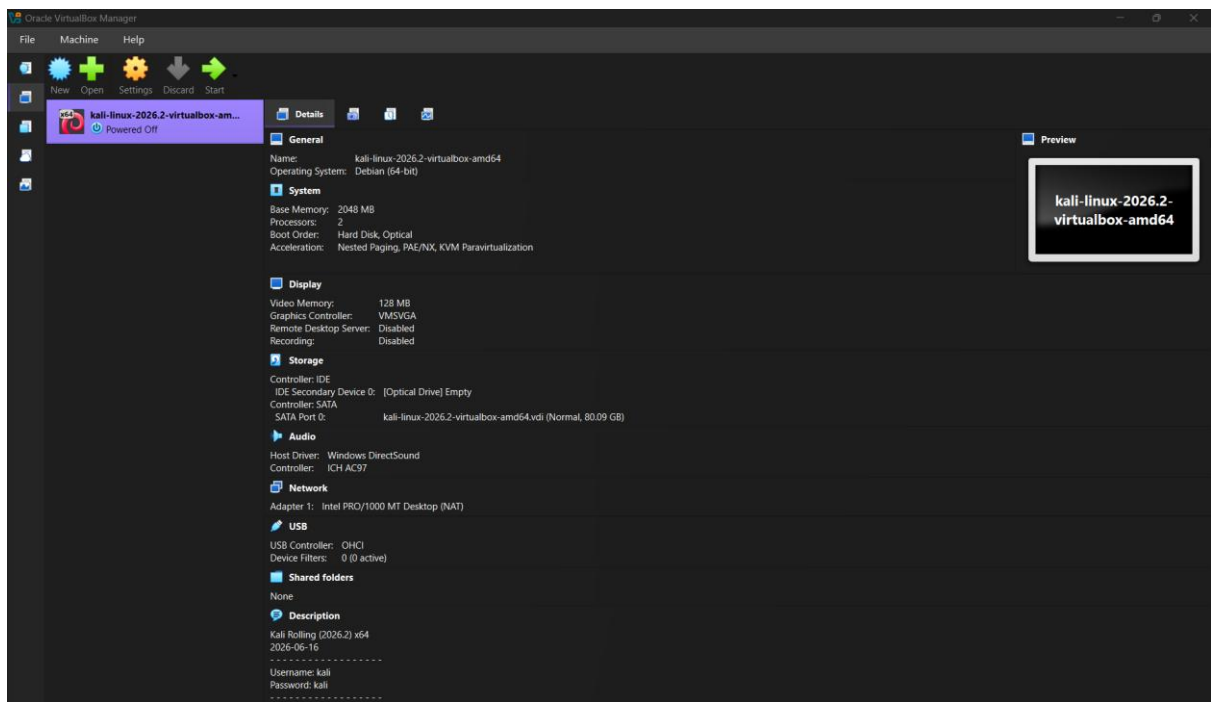


Step-5: Open Virtual box

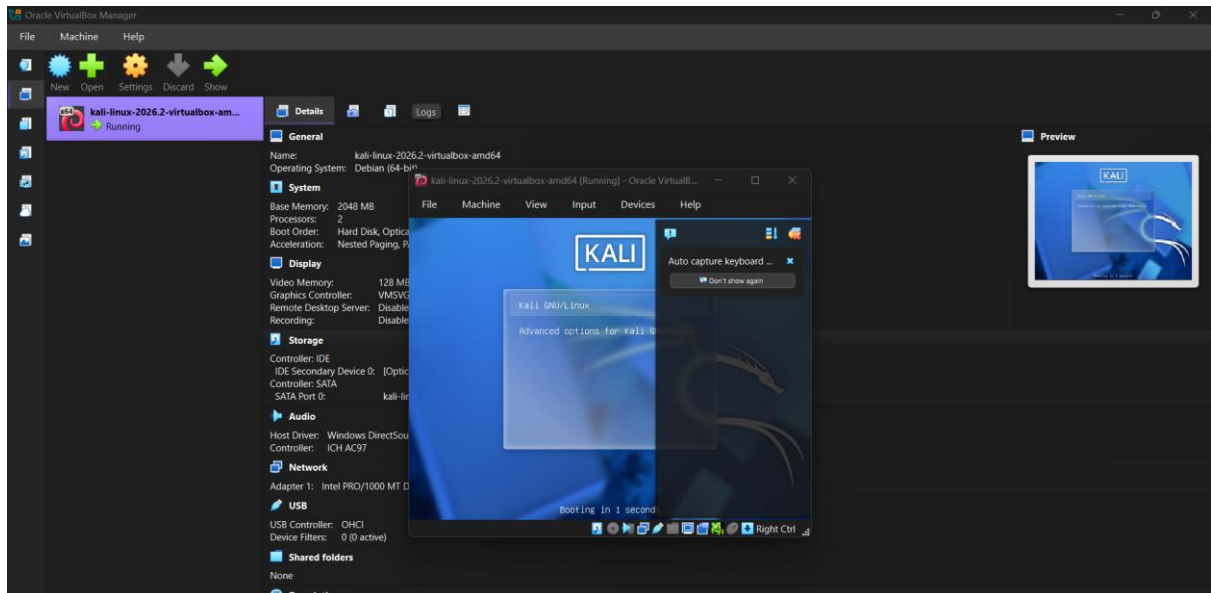


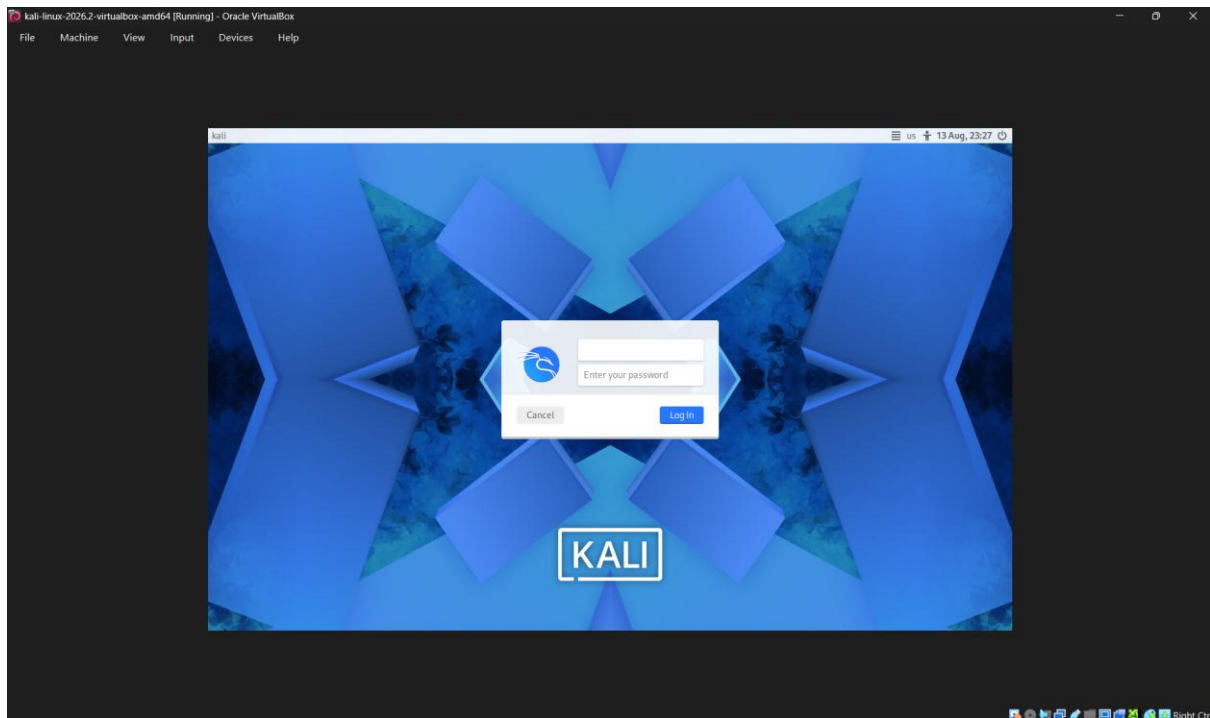
Step-6: Open the “saved VM”





Step-7: Start the VM





Thus, the installation of VMM (Hypervisor Type-2 Oracle VirtualBox) & VM (Kali Linux) is completed.

Step-8: Open terminal and create the following simple program, compile and execute to understand the working of the environment.

B) **Working with Linux Commands**

Execute the following commands in your environment, take output snippets, and provide a brief explanation.

- Terminal basics: pwd, mkdir, rmdir, dir, ls, cd, cd .., clear, history, man, whoami
- Files: vi, file, rm, cp, mv, cat, more, less, head, tail, chmod
- Redirection: >, >>, <

```
(kali㉿kali)-[~]  
$ pwd  
/home/kali  
  
(kali㉿kali)-[~]  
$ mkdir task3  
  
(kali㉿kali)-[~]  
$ cd task3  
  
(kali㉿kali)-[~/task3]  
$ vi program.c  
  
(kali㉿kali)-[~/task3]  
$ more program.c  
#include<stdio.h>  
int main(void)  
{  
    printf("Hello, World!\n");  
    return 0;  
}  
  
(kali㉿kali)-[~/task3]  
$ ls  
program.c
```

```
kali-linux-2026.2-virtualbox-amd64 [Running] - Oracle VirtualBox  
File Machine View Input Devices Help  
  
kali@kali: ~/task3  
Session Actions Edit View Help  
#include<stdio.h>  
int main(void)  
{  
    printf("Hello, World!\n");  
    return 0;  
}
```

```
(kali㉿kali)-[~/task3]  
$ vi program.c  
  
(kali㉿kali)-[~/task3]  
$ gcc -save-temps program.c  
  
(kali㉿kali)-[~/task3]  
$ clear
```

```
(kali㉿kali)-[~/task3]
$ history
1  pwd
2  mkdir demo1
3  dir
4  clear
5  pwd
6  mkdir session1
7  dir
8  cd session1
9  cd demo1
10 clear
11 pwd
12 mkdir session1
13 dir
14 cd session1
15 cd .. demo1
16 clear
17 pwd
18 cd ..
19 cd session1
20 dir
21 # include<stdio.h>
22 clear
23 pwd
24 vi task1.c
25 more task1.c
26 cc task1.c
27 pwd
28
```

```
(kali㉿kali)-[~/task3]
$ whoami
kali
```

```
(kali㉿kali)-[~/task3]
$ rm program.c
```